

UNIDAD EDUCATIVA BOLIVAR

NOMBRE: Kevin Alexander Verdesoto Azogue

FECHA: 19/05/2026

CARRERA: Software

- **Ejercicio 1: Búsqueda Lineal de Productos**

Desarrollar un programa en C++ o Java que permita buscar un producto dentro de un arreglo utilizando el algoritmo de búsqueda lineal.

Indicaciones:

Crear un arreglo con 5 nombres de productos.

Crear otro arreglo paralelo con sus precios.

Solicitar al usuario el nombre del producto a buscar.

Recorrer el arreglo usando búsqueda lineal.

Mostrar:

Nombre del producto.

Precio correspondiente.

Utilizar break cuando se encuentre el producto.

Mostrar un mensaje si el producto no existe.

Análisis

Vamos a hacer que un programa busque en el arreglo el producto solicitado y su precio respectivo en sus definidas posiciones por lo que usaremos un ciclo for para buscar en todo el arreglo.

Algoritmo

Inicio del programa.

Crear y llenar los datos: Se crean dos listas (arreglos) de 5 elementos cada una. La primera guarda los nombres de los productos ("Manzana", "Pan", etc.) y la segunda guarda sus respectivos precios (\$0.50, \$1.20, etc.).

Inicializar bandera: Se crea una variable lógica llamada encontrado y se define como *Falso* (ya que aún no hemos buscado nada).

Pedir datos al usuario: Se muestra un mensaje en pantalla solicitando el nombre del producto y se guarda lo que el usuario escriba en una variable llamada producto_buscar.

Buscar el producto (Ciclo): Se recorren las listas posición por posición (desde la 1 hasta la 5):

- **Si** el producto en la posición actual es igual al que el usuario busca:
 - Se muestra en pantalla el nombre del producto y su precio.
 - La variable encontrada cambia a *Verdadero*.
 - Se interrumpe el ciclo (ya no tiene sentido seguir buscando).

Verificar resultado: Al salir del ciclo, se revisa la variable encontrado. Si sigue siendo *Falsa*, significa que el producto no existía en la lista y se muestra el mensaje "Producto no encontrado".

Fin del programa.

Código en C++

```
#include <iostream>
#include <string>

int main() {
    std::string products[5] = {"Manzana", "Pan", "Leche", "Arroz", "Azucar"};
    double prices[5] = {0.50, 1.20, 0.80, 2.00, 1.50};

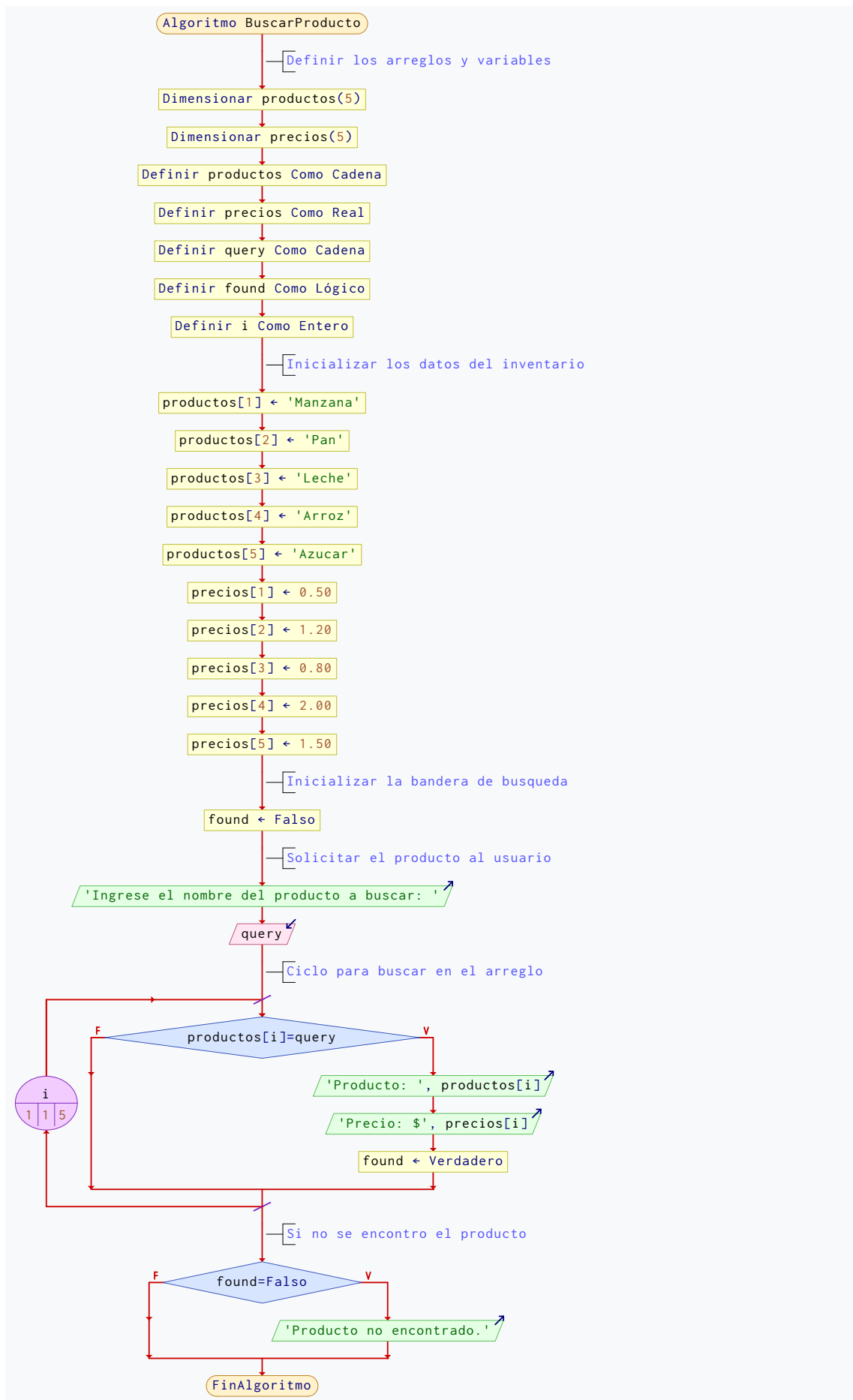
    std::string query;
    std::cout << "Ingrese el nombre del producto a buscar: ";
    std::getline(std::cin, query);

    bool found = false;
    for (int i = 0; i < 5; ++i) {
        if (products[i] == query) {
            std::cout << "Producto: " << products[i] << "\n";
            std::cout << "Precio: $" << prices[i] << "\n";
            found = true;
            break;
        }
    }

    if (!found) std::cout << "Producto no encontrado." << std::endl;

    return 0;
}
```

Diagrama de flujo



Casos de Prueba

```
Ingrese el nombre del producto a buscar: Pan
Producto: Pan
Precio: $1.2
Ingrese el nombre del producto a buscar: Leche
Producto: Leche
Precio: $0.8
```

- Ejercicio 2: Prevención de Buffer Overflow

Realizar un programa en C++ o Java que muestre los elementos de un arreglo sin provocar desbordamiento de memoria.

Indicaciones

Crear un arreglo con 6 números enteros.

Utilizar un ciclo for para recorrer el arreglo.

Validar correctamente los límites del ciclo.

Mostrar cada posición y su valor.

Agregar un comentario explicando qué ocurriría si el ciclo supera el tamaño del arreglo.

Análisis

Vamos a crear un código simple en el que se va a ir recorriendo el arreglo y se van a imprimir los valores de este mismo.

Algoritmo

Inicio del programa.

Creación del arreglo: Se reserva espacio en memoria para una lista (arreglo) de 6 números enteros y se le asignan los valores iniciales: 10, 20, 30, 40, 50, 60.

Inicio del recorrido (Ciclo desde 0 hasta 5): Se utiliza una variable contador (índice i) que empieza en 0.

Evaluación de la condición: Mientras el índice i sea menor que 6 (es decir, de 0 a 5), se realizan las siguientes acciones:

- Se muestra en pantalla el texto "Posicion ", seguido del valor actual de i, y luego el número almacenado en esa posición del arreglo (numeros[i]).
- Se incrementa el índice i en 1 unidad (++i).

Fin del ciclo: Cuando i llega a 6, la condición ($i < 6$) ya no se cumple y el ciclo termina, evitando salir de los límites de la memoria reservada.

Fin del programa.

Código en C++

```
#include <iostream>
int main() {
    int numeros[6] = {10, 20, 30, 40, 50, 60};

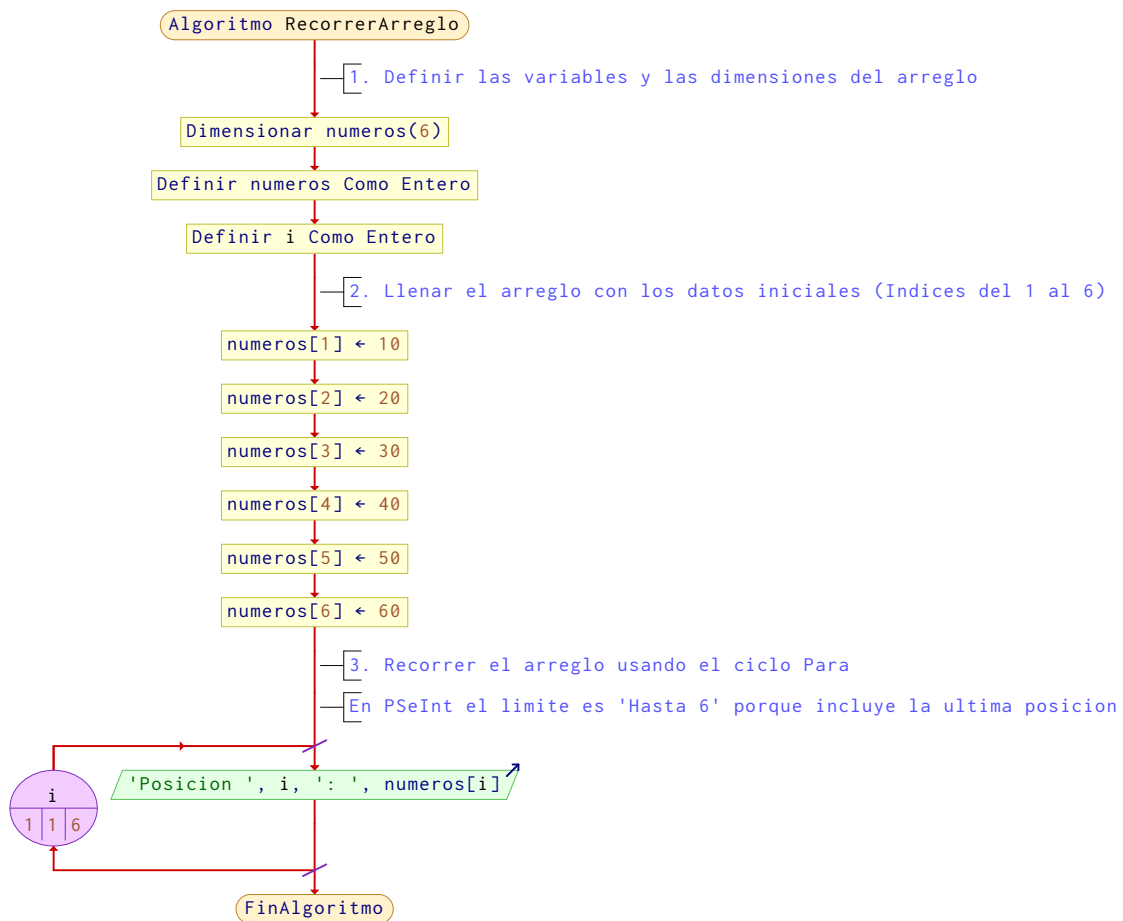
    // Recorremos el arreglo usando un ciclo for.
    // El límite correcto es  $i < 6$ , porque el arreglo tiene 6 elementos.
    for (int i = 0; i < 6; ++i) {
```

```

std::cout << "Posicion " << i << ": " << numeros[i] << std::endl;
}
// Si el ciclo supera el tamaño del arreglo (por ejemplo i <= 6 o i < 7),
// el programa intentaría acceder a memoria fuera del arreglo.
// Esto puede causar un comportamiento inesperado, datos incorrectos o falla del programa.
return 0;
}

```

Diagrama de flujo



Casos de Prueba

```

Posicion 0: 10
Posicion 1: 20
Posicion 2: 30
Posicion 3: 40
Posicion 4: 50
Posicion 5: 60

```

Link del Github

<https://github.com/Kevin-Verdesoto34/Operaciones-Avanzadas-e-Integridad-de-Datos>